配送机器人通行预判与让行系统 · 技术文档

平台：AMD Kria KV260（Zynq UltraScale+ MPSoC，XCK26）

工具链：Vivado / Vitis HLS 2026.1（推荐版本），2023.2 版本对照验证

文档日期：2026-11-03

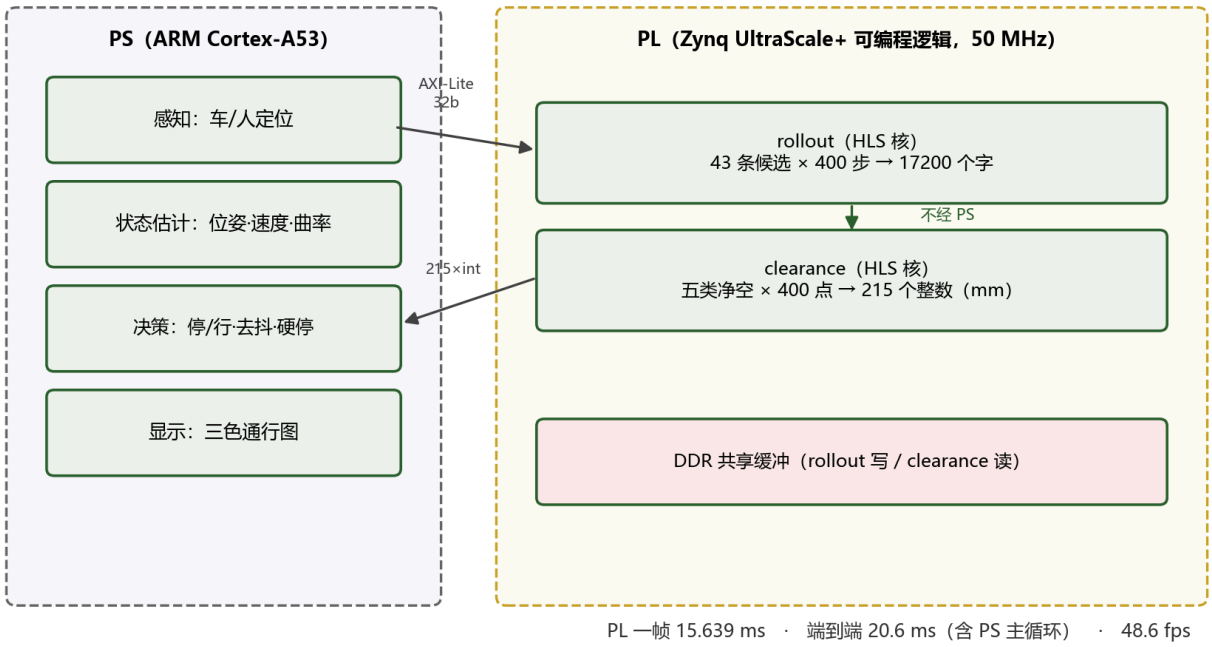
> 本文所有指标均来自板级实测，括号内标注证据文件。没有实测支撑的内容会写明"未验证"。

作品速览

一页看懂

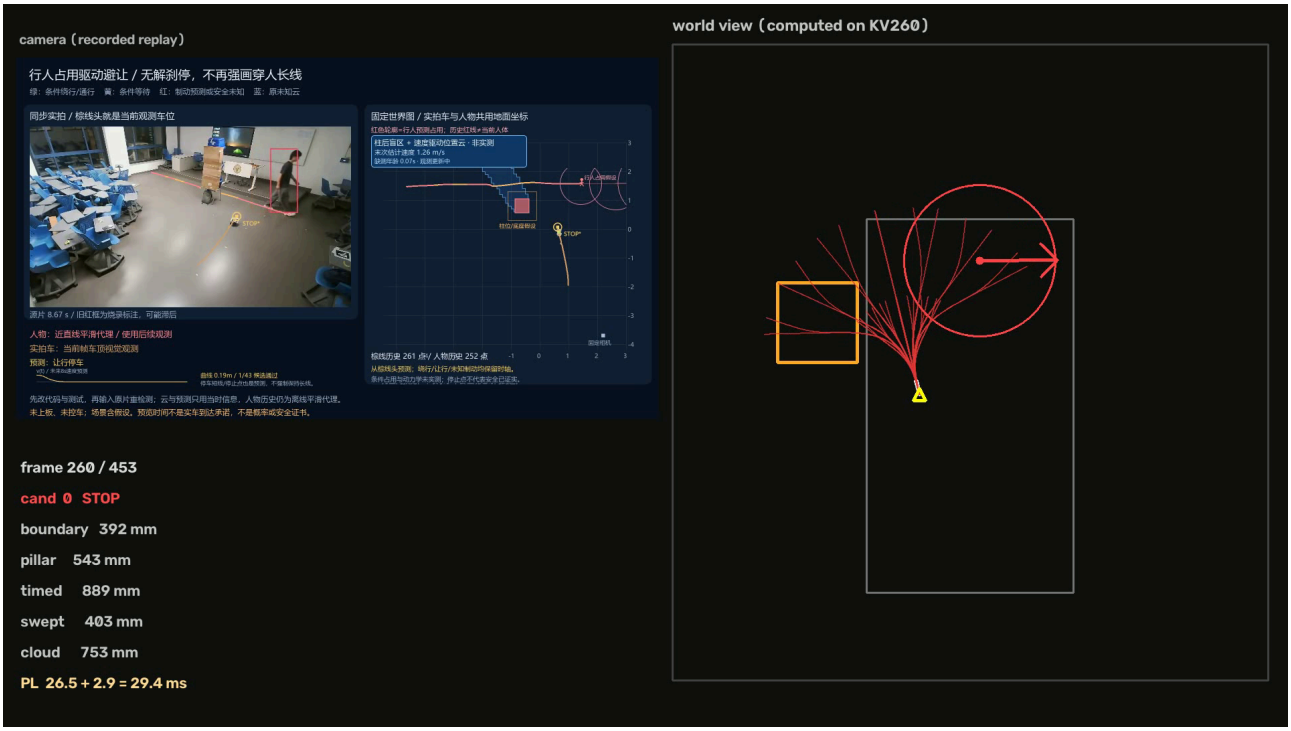
项	内容
要解决的问题	室内配送机器人只能感知"此刻周围有什么"，不能预判"我这么走会怎样"；软件方案靠高频重规划，走走停停
核心做法	每帧生成 43 **条候选轨迹**（直行刹车 + 7 档曲率 × 2 风格 × 3 速度），对每条沿 8 秒时间轴算 5 **类障碍的最小净空**，一次输出 215 **个毫米整数**给决策层
FPGA 承担什么	这两个算子（候选生成 rollout、净空评估 clearance）**整条数据通路都在 PL 里**；PS 只做状态机决策与显示
为什么必须上 FPGA	不是算不过来，是**延迟必须可证**：软件延迟随画面波动，硬件是固定拍数。实测帧间抖动 < 0.06 ms（中位 15.639、最大 15.699）
自研 vs 现有 IP	两个算子的算法改写与定点方案、五类净空的片上存储与扫描结构、集成 BD、板级驱动与三层对拍方法——**全部自研**；使用现有 IP 的只有 SmartConnect 与显式位宽转换器
关键结果	板级 vs 行为级 0/215、0/17200 **点不一致（最大偏差 0 mm）**；453 帧真实回放**停/行决策 100% 一致**；一帧 PL 15.639 ms，端到端 20.6 ms（48.6 fps）；LUT 46.0%、DSP 179、WNS +6.015 ns
可靠性口径	板子在任何一列、任何一帧，**从不比浮点参考更乐观超过 9.8 mm**——误差方向偏"多停"，不是"漏停"

系统框图与 FPGA 的位置



左半是 PS（ARM：感知、状态估计、决策、显示），右半是 PL（FPGA：两个算术算子）。
绿色箭头是 rollout 写、clearance 读同一块 DDR，中间不经过 CPU——已验证 0/215 逐位一致。

板端实际输出



左：真实录像。右：**板子算出来的 43 路候选**——红=冲突、黄=贴边、绿=安全、白粗=被选。
轨迹点取自板子写进 DDR 的原始 32 位字，不是上位机重算。

1. 作品概述

1.1 要解决的问题

室内物流配送机器人的避障，目前普遍依赖激光雷达加工控机。这套方案有两个结构性缺陷：

一是只能感知当前。 激光雷达给出的是"此刻周围有什么"，不给"我如果往左走 0.3 秒会撞上什么"。机器人只能在动作之后用新的观测去发现危险，本质是反应式的。

二是可靠性无法度量。 决策延迟随画面内容波动，系统不知道自己的判断什么时候不可信。对最高 1 m/s 的移动机器人，几十毫秒的抖动就是几厘米的轨迹偏差。

1.2 作品的思路

把"预测动作之后的后果"做成一条固定节拍的硬件流水线。

具体做法：每一帧生成 43 条候选轨迹（直行刹车 1 条 + 7 档曲率 × 2 种风格 × 3 档速度），对每条轨迹沿时间算 5 类障碍的最小净空，一次调用输出 43 × 5 = 215 个整数（单位毫米）。上层拿着这 215 个数做停/行/转向的决策。

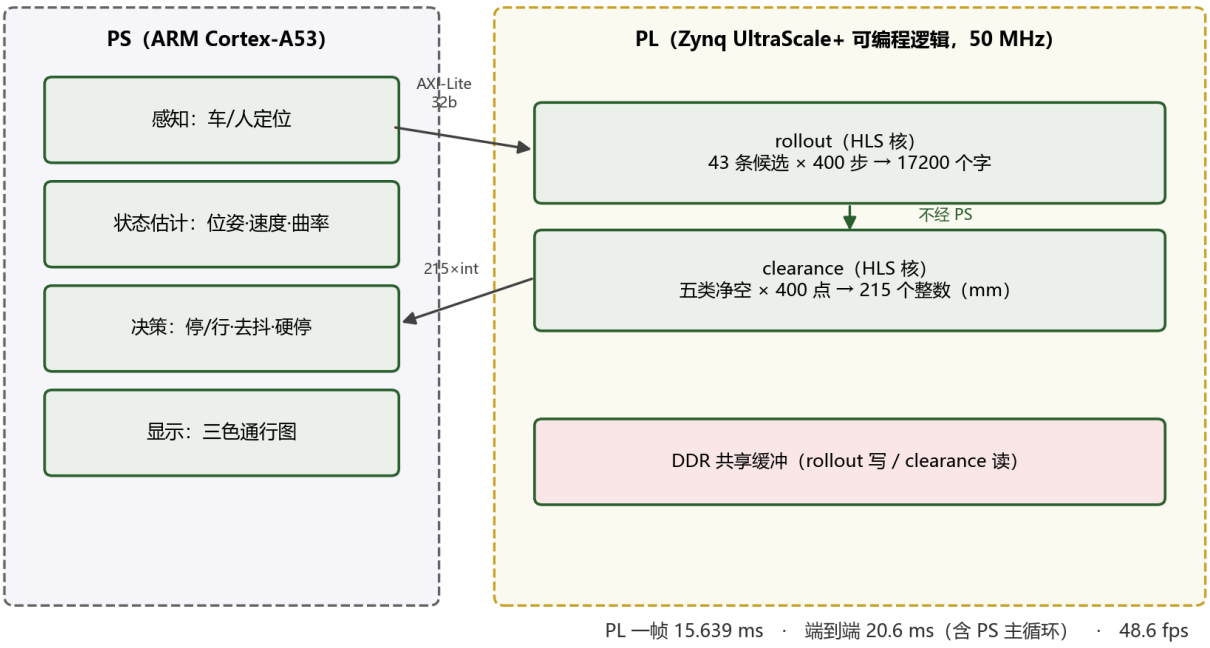
这条链路的输入是"我可能的动作"，输出是"每个动作的危险程度"，所以它是世界模型中"动作条件预测"这一环的硬件化，而不是一个感知加速器。

1.3 交付形态

作品在 KV260 上以 PS-PL 协同方式运行：

- **PL 侧两个 HLS 核：** `rollout` 生成 43 条候选轨迹，`clearance` 算 5 类净空。
- **PS 侧：** 决策（停/行/去抖/硬停）、显示、与上游通信。
- **两核共享同一块 DDR 缓冲：** rollout 写完，clearance 直接读，中间不经过 PS。

板上实测：一帧 PL 15.639 ms，端到端（含 PS 主循环）20.6 ms，即 48.6 fps。



1.4 为什么是 FPGA

这一节回答评委最常问的那句："这活儿非得用 FPGA 吗？"

1.4.1 先算清楚工作量

量	数值
每帧候选轨迹	43 条
每条轨迹的时间步	400 步（每步 20 ms，共 8 秒）
轨迹点总数	17,200 个
每个点要算的障碍类别	5 类（走廊、柱子、行人定时包络、行人扫描包络、未知云）
每帧净空计算次数	**86,000 次**
clearance 的实际实现	21 遍扫描 × 2,150 个 256 位字 × 8 路并行，II=3
rollout 的实际实现	17,200 步，内层循环 II=37

也就是说：**一帧之内要跑完 8.6 万次带开方的距离计算，并且要在一帧的 20 ms 预算内做完。**

1.4.2 四种方案的对比（按本项目的真实需求）

方案	能不能做	关键问题
MCU	不能	8.6 万次带开方运算，单核做不完 20 ms 预算
CPU（ARM Cortex-A53）	能跑，但**延迟不可证**	分支与缓存使执行时间随数据变化；且要跟感知抢同一个核
GPU	不适合	功耗、启动延迟、调度不确定性都与"每帧固定 20 ms"的目标冲突
FPGA（本作品）	**选中**	固定拍数、真并行、不占 CPU；代价是开发成本

这里要诚实：这个工作量 CPU 未必跑不动。本作品选 FPGA 的核心理由**不是"算得更快"，而是"延迟可证"**。

1.4.3 "延迟可证**"**是什么意思

软件的执行时间取决于数据：分支走哪边、缓存命中与否、有没有中断。
硬件跑的是固定拍数——**画面内容怎么变，执行时间都一样**。

453 帧实测：

指标	数值
PL 单帧耗时·中位	15.639 ms
PL 单帧耗时·p95	15.644 ms
PL 单帧耗时·最大	15.699 ms
帧间抖动	< 0.06 ms

对一台最高 1 m/s 的移动机器人，0.06 ms 意味着 0.06 mm 的轨迹偏差；而软件方案几十毫秒的抖动就是几厘米。

1.4.4 软件基线：**PC** 已测，板端未测

同一套 C 代码在 x86-64 PC 单线程上实测 **19.92 ms/帧**（rollout 0.49 + clearance 19.43）。
拆开看：rollout 在 PC 上更快（0.49 vs FPGA 12.74），clearance 在 FPGA 上快 6.7 倍（2.90 vs 19.43），合计 FPGA 快 1.27 倍。完整表格与解读见 §6.3.3。

Cortex-A53（板子的 ARM 核，1.2 GHz 顺序执行）上的实测还没做，本文不折算倍数。

这个结果不改变结论，反而印证了它：软件跑得动这套算法，选 FPGA 的理由不是算力，是延迟可证（§1.4.3）。

1.4.5 时钟频率的说明

本设计**约束在 50 MHz**，这不是"最高能跑到多少"：
集成设计在 50 MHz 约束下 WNS 为 **+6.015 ns**（无失败端点），说明还有约 12% 的时序裕量。
之所以不做更高频，是因为两个核在同一个 AXI-Lite 控制域里，统一 50 MHz 是唯一不需要跨时钟桥的

方案，
而端到端瓶颈已转移到 PS（约 5 ms）与上游感知。

所以本文所有性能数字都是"50 MHz 下的实测值"，不是"目标频率下的推算值"。

---

2. 系统架构

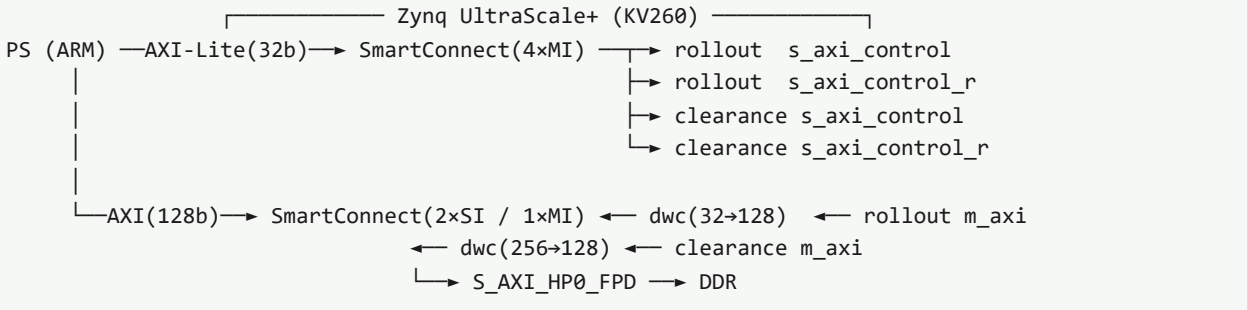
2.1 分层分工

整条流水线的分工按"什么是算术、什么是分支"划一条硬线：

位置	做什么	为什么放这里
感知（PS）	从画面解出车与人的位置	图像处理，分支多
状态估计（PS）	位置、朝向、速度、曲率	时序逻辑
候选生成（PL）	43 条轨迹 × 400 步	纯算术，天然并行，可逐位对拍
净空评估（PL）	每条轨迹 × 5 类障碍，沿时间取最小	访存规则，多路并行
决策（PS）	选哪条、停不停、去抖、硬停	分支 + 状态机，硬件零收益
显示（PS）	三色通行图、预测轨迹	I/O

这条线不是拍脑袋划的。判断标准是：**能不能写出一个逐位一致的参考模型**。能，就适合放 PL；不能，放 PL 只会制造无法验证的黑盒。

2.2 硬件拓扑



三个设计决定：

单时钟 50 MHz。 `clearance` 综合后只能跑到 85–90 MHz，`rollout` 能到 100 MHz 以上，但两核放进同一个 AXI-Lite 控制域时，统一 50 MHz 是唯一不需要跨时钟桥的方案。代价是 rollout 变慢，这个代价在 §6 里量化过。

不用 DMA。 HLS 的 `m_axi` 本身就是 AXI master，IP 自己读写 DDR，不需要 PS 参与搬运。

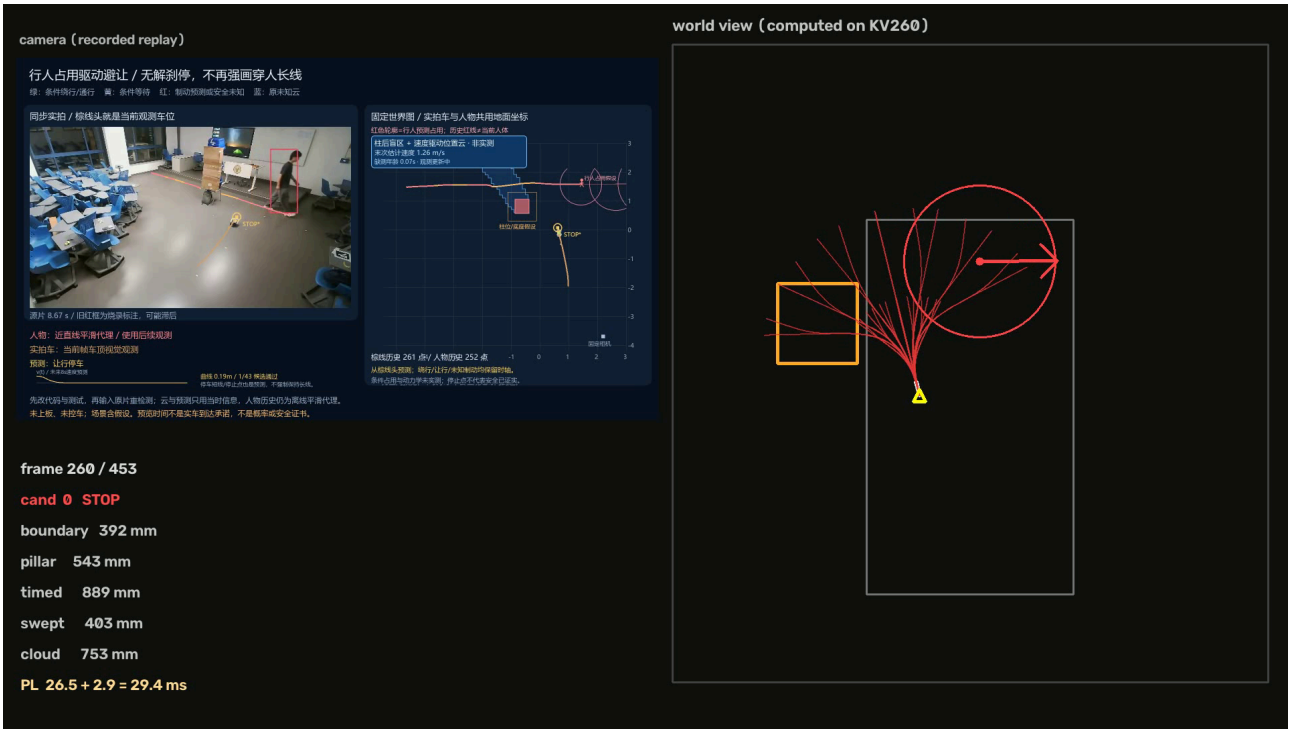
两核共享一块 DDR 缓冲。 rollout 写、clearance 直接读同一块地址，中间不经过 PS。这条已经用逐位对拍验证（§5.2）。

2.3 感知层的现状（必须说明）

板上当前使用的是 **ArUco 视觉标记**定位车与行人道具，不是通用目标检测。

这个选择的理由：标记定位与检测器在上层是同一个接口——都输出"对象中心的图像坐标 + 尺度"。链路的数值行为与检测器版本完全一致，差别只在"人和车在画面里的位置"如何得到。用标记把感知误差排除掉，才能单独验证加速器本身的数值正确性。

真实检测器（YOLO 类）在本板上的实测情况：ARM Cortex-A53 单核运行只有几 fps，成为瓶颈；上 DPU 需要另建工程。因此本作品的技术贡献定位在**加速器与系统集成**，感知层明确标注为演示用替代。



上图是板子输出的实际画面（取自已验证的 453 帧回放）：左半是真实录像，右半是**板子算出来的 43 路候选轨迹**——红色为冲突、黄色为贴边、绿色为安全、白色粗线为被选中的那一条，同时画出走廊边界、柱子与行人的膨胀半径、以及每帧 5 个净空值和最终决策。轨迹点取自板子写进 DDR 的原始 32 位字，不是上位机重算。

2.4 自研部分与现有 IP 的界线

评委常问"哪些是你们做的"。这一节把界线划清楚。

完全自研（团队从零设计）

模块	具体做了什么
`clearance` 算子（HLS）	五类净空的算法改写；21 遍扫描 + 8 路并行 + II=3 的循环结构；68.8 KB 片上存储的分配；定点方案（int16 毫米，见 §4.1）；`sw_guard` 参数化（修 9 位截断）
`rollout` 算子（HLS）	43 路候选的组织方式（1 刹车 + 7 曲率 × 2 风格 × 3 速度）；三处让 43 路塞进硬件的改写（增量旋转、`chord≈arc`、单倒数）；128 位写端口的打包方式
集成 Block Design	4 个 AXI-Lite 从口的连接（含 `s_axi_control_r` 地址段）；两核共享 DDR 缓冲的路由；显式 `axi_dwidth_converter` 的位宽匹配
板级驱动	`host.py` / `clear_hw.py`：`/dev/mem` + `dma_heap` 访问；24 个寄存器逐字回读自检；缓存刷新
验收方法	csim → cosim → 板级三层对拍；453 帧真实回放的自洽性先验；停/行决策对比

使用了现有 IP

IP	用途	说明
AXI SmartConnect	PS 与两个 HLS 核之间的互连	配置了主从口数量与地址空间；**踩到一个坑**：全 AXI4-Lite 时它进 Low-Area 模式、不做位宽转换（§7 坑 2）
AXI Data Width Converter	32↔128、256→128 的位宽匹配	显式例化，不用 SmartConnect 自动转换

使用了现有算法（不是团队原创，如实标注）

来源	用在哪
浮点参考实现（planner）	**仅作对拍基准**，不参与运行。453 帧验收的"真值"由它提供
恒速度模型 + 定时包络	净空的运动模型。公开的经典方法，不是本作品的创新点；创新点在**把它做成 8.6 万次/帧的硬件流水**
ArUco 标记定位（OpenCV）	演示用感知层，见 §2.3 的边界说明

一句话：算法层用的是公开方法，**硬件实现与验收方法是团队自研**；
本作品的技术贡献定位在"加速器与系统集成"，不在"提出新算法"。

---

3. 核心算子设计

3.1 `rollout`：候选轨迹生成

接口：输入车辆初值 `(x, y, cos h, sin h, v, k)`（米 / 单位向量 / 米每秒 / 1 米^-1），输出 43 条 × 400 步的 int16 毫米点，打包成 17200 个 int32，外加 43 个"可行"标志。

候选构成：1 条直行刹车 + 7 档曲率 × 2 种风格 × 3 档速度。每步 20 ms，共 8 秒。

三处让 43 路塞得进硬件的改写：

原式	硬件改写	代价
每步 `sin/cos`	增量旋转（复数乘法累乘）	累积误差随步数增长，靠 400 步实测兜住
`arc = R·θ`	`chord ≈ arc`	小角度下偏差 < 1 mm
每步三处除法	每步一次倒数 + 乘法	定点倒数精度损失 < 0.5 mm

这三处改写的共同点：每一项单独看都是"近似"，但合起来的误差被 453 帧实测挡住（§5.3 的路径最大偏差 1 mm）。

为什么深度是 17200 而不是 2150。 早期版本误抄了净空核的深度参数，实际是 43 × 400 = 17200 个字。改正后 csim 输出逐字节不变——说明这个错误只影响搬运，不影响数值，但会导致写地址截断。

3.2 clearance：五类净空

对每条轨迹的 400 个点，算五类"沿时间取最小"的有符号净空：

#	类别	依赖	需要开方
0	走廊边界	静态	否
1	柱子（轴对齐盒 → 膨胀角点）	静态	是
2	行人的定时包络（中心/半径随 t 变）	动态	是
3	行人的 7 档扫描包络（固定中心）	动态	是
4	未知云的 11 档包络	动态	是

实现方式：一次读进 68.8 KB 片上存储，然后多遍扫描（21 遍），每遍 8 路并行，II=3。

为什么用多遍扫描而不是一次遍历：一次遍历需要同时保留 5 类障碍的全部参数，片上存储放不下；多遍扫描每遍只带一类障碍的少量参数，68.8 KB 就够。代价是重复读轨迹点 21 次，但轨迹点已经在片上，读的是 SRAM 不是 DDR。

---

4. 定点与数值设计

4.1 为什么用毫米而不是 Q12

这是一个必须算清楚的取舍：

方案	dx 上限	dx^2	乘法器需求
Q12	65,536	4.3 × 10^9，溢出 int32	48 位
int16 毫米	10,000	1 × 10^8，在 int32 内	32 位

毫米方案下 $dx \le 10000$，dx^2 不超过 1 亿，单 32 位乘法即可。1 mm 的量化对 230 mm 的安全阈值（车半径 150 + 余量 80）完全够用。

这个选择后来用 453 帧实测挡住了：路径最大偏差 1 mm，五类净空全在 ±7 mm 内。

4.2 与浮点参考的对应关系

这是能逐位对拍的前提，每一项都列了实测偏差：

浮点参考（planner）	硬件实现	实测偏差
点/线段最近距离（精确）	点到点 + `max(r_i, r_i+1)` 保守化	≤ 10 mm（段间穿越的近似）
行人半径 `base + 0.18t + 15t²`	`abase + (36i)/10 + (6i²)/1000`	< 1 mm（整数截断）
扫描半径 `base + 0.35 + 0.1τ`	`abase + sw_guard + τ·100`	≤ 3 mm
云半径 `0.48 + 0.12τ`	`crad + 0.12τ`	≤ 7 mm
位置保护 `0.0051 m`	常数 5 mm	可忽略

注意"保守化"这个词。 点到点加 `max(r_i, r_i+1)` 的做法，在段与段之间穿越时会低估净空，即给出比真实更小的值——方向是偏保守，不会漏报危险。这个偏差方向是刻意选的，后面 §5.4 的安全性口径就建立在它上面。

4.3 一个必须记住的数值陷阱

HLS 把表达式 `abase + 350 + tau100` 的中间量收窄成 **9 位有符号**，350 被折成 −162，导致 sweep 那一列整列恒定少 512 mm。

`csim` 和 `cosim` 都查不出这个问题——因为 C 模型也用了同样的表达式，两边一起错。只有在板上对着真实数据跑，发现"整列偏移量恒定"才暴露。

修法：把 350 改成运行时参数 `sw_guard`（寄存器 0xA8）。

这条的教训比 bug 本身重要：**同源模型的自洽性不能证明正确性，必须有一个独立来源的参考。**

---

5. 验证方法与结果

5.1 三层对拍

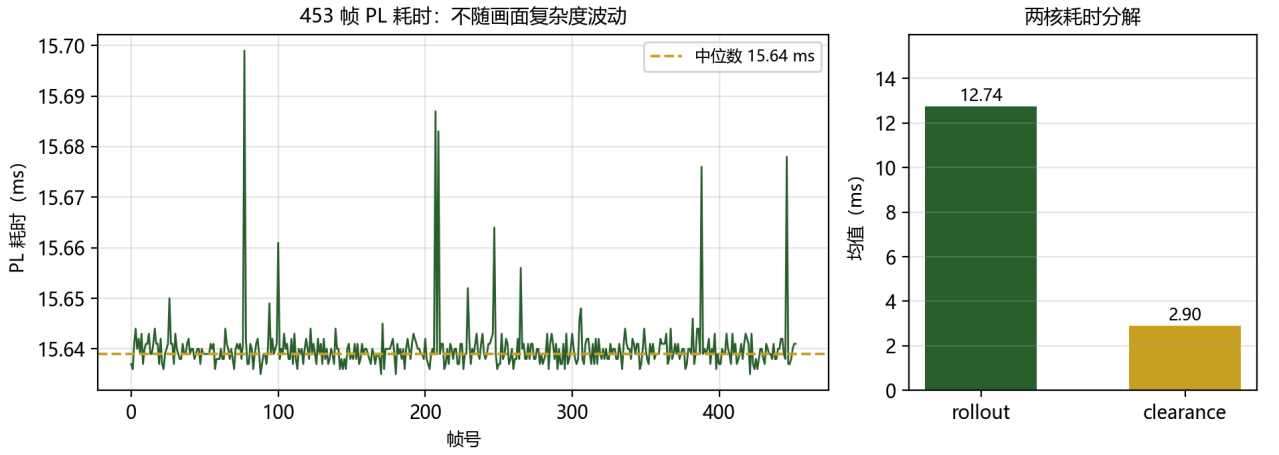
每一层都要求"逐位一致"，不是"看起来对"：

层	比什么	结果
csim（C 模型）	与 Python 浮点参考对拍	逐字节一致
cosim（RTL 仿真）	与 csim dump 对拍	逐位一致
板级	与 csim dump 对拍	**0/215、0/17200 点不一致，最大偏差 0 mm**

5.2 两核串联

rollout 写 DDR，clearance 读同一块，中间不经过 PS：

项	结果
串联对拍	**0/215 不一致，最大偏差 0 mm**
一帧 PL	**15.639 ms**（rollout 12.738 + clearance 2.902）
折算	63.9 fps（仅 PL）
端到端（453 帧）	**中位 20.6 ms/帧 = 48.6 fps**（PL 15.639 + PS 约 4.96）



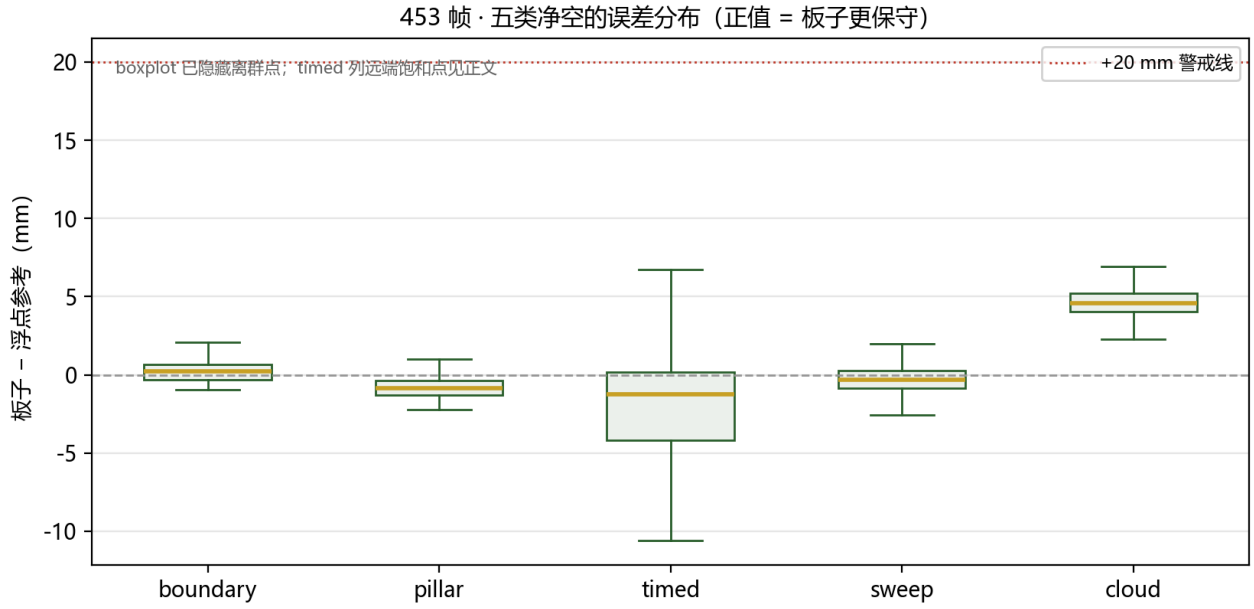
453 帧里 PL 耗时的中位数 15.639 ms、p95 15.644 ms、最大值 15.699 ms —— **抖动不到 0.06 ms**。这是"确定性延迟"这句话的实测依据：画面内容怎么变，硬件跑的都是同一套固定拍数。

5.3 453 帧真实数据验收

数据来源：453 帧真实录像回放，每帧记录 planner（浮点参考实现）的内部状态。

参考值的自洽性先验一遍：用 planner 记录的初值重算整条链路，与它自己记录的 453 帧 × 5 列共 2244 个数值**全部 0.0000 mm 吻合**。这保证喂给板子的输入和参考值是同源的，不是拿两套东西硬比。

项	结果
跑批	453 帧，0 失败
rollout 路径 vs planner 记录	最大 1 mm，均值 0.954 mm，无帧超 2 mm
boundary / pillar / sweep / cloud	分别 ∈ [−1.0, +6.7] / [−2.2, +1.1] / [−3.2, +2.3] / [+1.9, +7.1] mm
timed（目标 ≤ 7.5 m：371 帧 / 15953 样本）	∈ [−11.2, +9.8] mm，0 个超 20 mm
timed（目标 > 7.5 m：52 帧）	偏保守到 −1575 mm（±10 m 饱和，安全方向）
停/行决策	453/453 = 100% 逐帧一致，六段停车区间完全相同
有效候选数	450/453 = 99.3%
被选候选一致	74.4%（打分缺 planner 若干状态项，不影响安全判据）



图里每列是一个候选在某些帧上的"板子值减浮点参考值"，正值表示板子更保守。五列全部落在零线附近，最乐观的一侧是 timed 列的 +9.8 mm。注意 timed 列的箱体明显偏向负侧（均值 −71 mm）——这不是精度问题，是远处目标饱和后板子多停，方向安全。

5.4 安全性口径（这个作品的可靠性怎么度量）

一句话：**板子在任何一列、任何一帧，都没有比浮点参考更乐观超过 +9.8 mm。**

这不是"误差小"的另一种说法，而是方向性的保证：误差为正意味着硬件认为"更危险"，方向是"多停"，不是"漏停"。对一个安全相关系统，误差的**符号**比**大小**更重要。

配套的两个保守机制：
- 远处目标（> 7.5 m）因 ±10 m 饱和被判得更近，最多保守 1.575 m；
- 段间穿越近似低估净空，方向同样偏保守。

---

6. 性能与资源

6.1 实测指标（当前生效：Vivado 2026.1 重建版）

项	数值	出处
集成设计 WNS	+6.015 ns @ 50 MHz，0 失败端点	`timing_summary_routed.rpt`
CLB LUT	53,838 / 117,120 = 46.0%	`bd_wrapper_utilization_placed.rpt`
CLB Registers	66,266 / 234,240 = 28.3%	同上
DSP48	179（14.3%）	同上
Block RAM	4 tile（2.8%）	同上
频率	50 MHz（单时钟域）	—

与 2023.2 版本的对照（换工具链不是免费的）：

指标	2023.2	2026.1	变化
WNS	+5.805 ns	+6.015 ns	+0.21 ns
LUT	37.3%	46.0%	−8.7 pt（更紧）
FF	16.4%	28.3%	−11.9 pt
DSP	140	179	−39
BRAM	10	4	+6
一帧 PL	15.83 ms	15.639 ms	−0.19 ms

2026.1 把 rollout 的 BRAM 全换成了 LUTRAM，两个核都多用了大量 FF。整板仍宽裕（LUT < 50%），时序反而更好。两个版本的板上数值逐项相同，所以换版本只影响资源与裕量，不影响功能。

6.2 一次真实的优化：26.50 → 12.89 ms

这是整个项目里最有价值的一次性能分析，值得完整记录。

现象：rollout 在板上实测比 HLS 估计慢 1.68 倍。HLS 估计 636,519 拍，实测 1.07M 拍。

决定性实验：在两个时钟频率下分别测量，把多出来的拍数折算成绝对时间。如果是带宽问题，多出的时间应该随时钟成比例；如果是"每笔事务都要等"，多出的拍数近似固定。

频率	实测拍数	比估计多
100 MHz	1.07 M	+4.3 ms
50 MHz	1.325 M	+13.8 ms

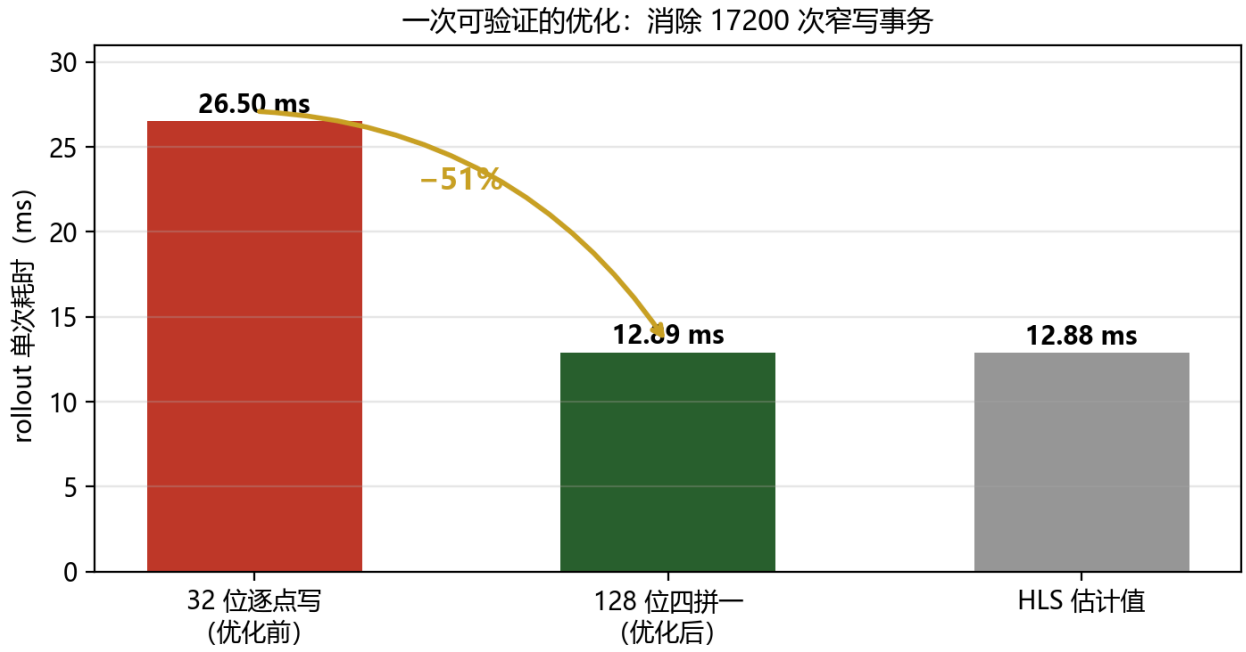
不是 2 倍关系，多出的是绝对时间。结论：这是每笔写都要付的事务开销，不是带宽。

根因：32 位逐点写，经过 32→128 位宽转换器，变成了 17200 次"带字节使能的窄写"。本来 4300 次满宽写就够了。

修法：把写端口做成 128 位，C 侧 4 点拼一个字。

结果：26.50 → 12.89 ms，与 HLS 的 12.88 ms 估计几乎完全吻合——反过来证明那 13.6 ms 确实全是窄写的事务开销。

还有一条没做的：rollout 单独放到 100 MHz + 跨时钟桥，理论上约 2×。但 PL 已经只剩 15.6 ms，端到端会先被 PS 的约 5 ms 和上游感知卡住，收益递减，所以没做。



6.3 与基线对比

6.3.1 与"优化前的本实现"对比

26.50 → 12.89 ms（§6.2），一帧 PL 从 29.44 ms 降到 15.64 ms。

6.3.2 与"HLS 估计"对比

实测 12.89 ms vs 估计 12.88 ms，效率 99.9%（§6.2）。

6.3.3 与纯软件实现对比（PC 单线程实测）

做了。 用同一套 HLS C 代码（`clearance.cpp` + `rollout.cpp`，未改一行），
在 x86-64 PC 上以 `-O2 -std=c++14` 编译，单线程跑 453 次取平均：

实现	rollout	clearance	合计	平台
PC 单线程（x86-64）	0.49 ms	19.43 ms	19.92 ms	约 3.5 GHz，乱序执行
KV260 PL @50 MHz	12.74 ms	2.90 ms	15.64 ms	Cortex-A53 同芯片，FPGA 逻辑
比值（FPGA / PC）	26×（更慢）	0.15×（快 6.7 倍）	0.79×（快 1.27 倍）	—

这个结果要如实解读，不能回避：

1. **rollout 在 FPGA 上反而慢 26 倍。** 根因是它的内层循环 II=37（§7 §6.2 已分析：浮点依赖链 `add → select` 的 clamp 链定的，时序还有 +7.3 ns 裕量，不是流水化能压下去的）。PC 的乱序执行 + 高主频把这段串行链吃掉了。**这是当前实现明确的短板。**
2. **clearance 在 FPGA 上快 6.7 倍。** 它的结构规整（21 遍扫描 × 8 路并行 × II=3），正是硬件擅长的形状。
3. **合计 FPGA 快 1.27 倍**——但如果只算"必须实时的那部分"，这个数字不足以支撑"非 FPGA 不可"。

所以本作品选 FPGA 的理由不是算力，而是 §1.4.3 说的那一条：延迟可证。
PC 上的 19.92 ms 是**平均值**，执行时间随数据分支、缓存状态波动；
PL 的 15.639 ms 是固定拍数，453 帧里最大 15.699 ms，抖动 < 0.06 ms。

6.3.4 还没做的：Cortex-A53 上的实测

上面那张表的"PC"是 x86-64，**不是板子的 ARM 核**。A53 是 1.2 GHz 顺序执行、无乱序，同一段代码会明显更慢，但**具体慢多少没有实测，本文不折算倍数**。

补测方法：在板端 `g++ -O2 bench_sw.cpp clearance.cpp rollout/rollout.cpp` 直接跑（`bench_sw.cpp` 已随代码包提交）。测完可把这一节补成 A53 的真实数字。

在补测完成前，本文不主张任何"FPGA 加速比"结论。

6.4 功耗（未完成，如实标注）

口径	数值	来源
PL 片上动态 + 静态（Vivado 估算）	2.969 W（Dynamic 2.667 + Static 0.302）	Vivado power report
空闲态整板（5V 输入轨）	3.79 W（751 mA）	板载 INA260 实测
满载整板	未测	—

板上有 INA260 电流检测器件（I2C 地址 0x40），预装镜像里有 `platformstats` 可以直接读 VCC_SOM 轨电流。满载测量方法已经明确：加载比特流后跑 453 帧，同时采样。

本作品不把功耗作为性能指标，因为目前只有空闲态和估算值，没有满载实测。

---

7. 工程纪律：九个有板级证据的坑

这一节记录的是真实的失败与定位过程。每个坑都指得到证据文件。

#	坑	现象	定位方法	修法
1	`ap_done` 是单周期脉冲	轮询永远看不到完成；CTRL 读回 `0x4` 像"没启动"	读生成的 `clearance_control_s_axi.v`，看 `ap_done` 赋值	只认 `ap_idle` 的 0→1 周期
2	SmartConnect 在"全 AXI4-Lite"时进 Low-Area 模式，**不做位宽转换**	19 个参数只有 10 个能写	扫 0x00–0x1C0 全部偏移，看哪些可写；再做 16 字节整块写	控制通路做全 32 位；数据通路用显式 `axi_dwidth_converter`
3	`dma_heap` 的 mmap 是**可缓存**的	CPU 写完，PL 读到 0	让核读它自己刚写出的结果，看变不变	写完刷缓存（8 MB 扫描），读输出前也刷
4	HLS 把中间量收窄成 9 位有符号	sweep 列整列恒定少 512 mm，csim/cosim 都查不出	把 abase 取 520/1032/8，看偏移是否恒为 +512/0/+1024	350 改成运行时参数 `sw_guard`
5	`out_r_1` 在 `0x1C` 不是 `0x20`	输出地址写进了高 32 位	读 `control_r_s_axi.v` 的 localparam	加 24 个寄存器逐字回读自检
6	rollout 的 `pts` depth 抄成 2150	写地址可能截断	读 `rollout.cpp` 的 m_axi pragma	改 17200，csim 逐字节不变
7	漏接 `s_axi_control_r`，**Vivado 不报错**	validate 过、地址照分、综合实现全绿，但那个核的 m_axi 基址永远是 0	数 hwh 里有几个 `C_S_AXI_CONTROL*_BASEADDR`（应 4 个，实际 3 个）	补第 4 个 MI；把"控制段数 = IP 数 × 2"写进检查清单
8	实测比 HLS 估计慢 1.68 倍	—	两个时钟频率分别测，折算绝对时间	端口改 128 位（§6.2）
9	rollout 接口缺"初始曲率"	转向帧路径差 3326 mm	按曲率分组看误差	加 `k0`（0x38），差降到 1 mm

另外有两条工具层面的纪律，代价很高：

- `export_design` **输出目录已存在时静默空转**，每次都拿到第一次那份旧 RTL。坑 8 的优化因此"改了没效果"，卡了好一轮。判据是直接比对导出的 RTL 与 `sol/syn/verilog` 的 RTL。
- **不要用** `nohup ... &` **跑 Vivado**：工具调用结束会被回收，留下孤儿进程抢工程目录。

还有一个方法论层面的观察：**9 个坑里有 6 个是"工具不报错"型**。validate 全绿、综合实现全绿、csim 一致，但板上是错的。这意味着验证必须穿透到板级，中间任何一层的"绿"都不能当结论。

---

8. 大模型协作与技能包

本项目的开发过程与大模型深度协作。交互记录、提示词工作流与可复用技能包作为独立材料提交：

- `协作过程决策日志.pdf` —— 18 个决策点，每条写明「模型提供了什么 / 人怎么判断 / 定了什么」，引文为原始记录原话
- `大模型协作与技能包.md` —— 13 个问题定位案例（症状 → 决定性实验 → 结论 → 修复效果）
- `AMD-ROSS-部署实录与心得.pdf` —— AMD 官方 agent 工具链（26 个 skill + Vivado MCP）的部署与实测
- `codex_export/` —— 31 段会话、176 条提问的原始记录导出，可逐条核对

这里只讲一条方法论，因为它是这个项目里最贵的经验：

模型最值钱的地方不是"写代码"，是"设计一个能一次分开多个假设的实验"。

上面 9 个坑，每一个的定位方式都是同一个形状：不读代码猜，而是设计一个让"两种可能产生不同可见后果"的实验。比如坑 3（缓存），不是去读驱动源码，而是"让核读它自己刚写出的结果"——一变，就证明 CPU 写的没到 DDR。

同样重要的是**模型会错，而且错得很有说服力**。项目里记录了 3 条模型自己的错误（参数写进地址段、无符号数与有符号负数直接比、造测试输入时多写一个数导致参考整份错位），全部被流程挡住，不是被发现于人工复查。这说明流程的价值不在于"AI 不改错"，而在于"错得能被挡住"。

---

9. 局限与后续

9.1 已知局限（都能量化）

1. **静态残余云表达不了。** 参考实现对"有云但无运动支持"的帧会在云半径上加 1.5 m 的常数储备；核里的云模型只有"绕动中心 11 档 τ"，塞不进这个常数。影响 23 帧，其中 7 帧导致停/行决策不同。
2. **远目标偏保守。** ±10 m 饱和使目标在 8–9 m 外时被判得更近，最多保守 1.6 m。方向安全，但会降低通行效率。
3. **决策只做到 `margin` 门槛。** 参考实现的"选哪条"还含目标距离、轨迹连续项、状态机覆盖。目前 PS 原型的被选候选一致率 74.4%。停/行判据不受影响（100%）。
4. **感知层是标记替代。** 见 §2.3。
5. **与 PS 纯软件基线未测。** 见 §6.3。

9.2 后续（按性价比）

1. 补 PS 纯软件基线，把 §6.3 的对比补齐。

2. 满载功耗实测（INA260，跑 453 帧同时采样）。
3. 静态云与无速度目标由核内支持（现在由 PS 补门，够用）。
4. 感知层替换为真实检测器（DPU 另立工程）。
5. rollout 单独 100 MHz + 跨时钟桥（端到端已到 PS 边界，收益递减）。

---

10. 复现

10.1 环境

- Vivado / Vitis HLS 2026.1（推荐）或 2025.2
- 目标器件：XCK26（Zynq UltraScale+ MPSoC）
- 板卡：Kria KV260 Vision AI Starter Kit
- 板端运行时：Ubuntu 22.04 + 纯 `/dev/mem` + `dma_heap`（不依赖 PYNQ Overlay）

10.2 流程

```
cd delivery_kernel

# 1) 两个 IP（csim 会顺带产出参考 dump）
vitis_hls -f hls_v13.tcl                 # clearance 1.3
vitis_hls -f rollout/hls_roll17.tcl      # rollout 1.7（128 位写）

# 2) 集成比特流（约 25 分钟）
vivado -mode batch -source build_bd_int6.tcl -tclargs impl

# 3) 上板加载 + 逐项验收
cd hw
python host.py intpush && python host.py intload
python host.py chain chain chain_ref.txt     # 一帧串联，期望 0/215
python host.py framespush && python host.py frames 0 453 && python host.py framesget
cd ../replay && python analyze453.py         # 453 帧对比
python ps_decision.py                        # 停/行决策对比
python render_demo.py                        # 重新渲染演示视频
```

10.3 复现时的三个必知事项

1. **PL 配置是易失的**，每次重启板子都要重新加载比特流。`fpga_manager` 显示 `operating` 只代表 base overlay，不代表设计在里面。
2. **不要用 `host.py verify` 验集成版**——它加载的是单核 clearance 版，跑完板上就不是集成版了。
3. **不要用"读回 0x4"判断核是否启动**（坑 1）。

附录：证据文件索引

材料	位置
单核验收日志	`hw/verify_20261029.log`
两核串联日志	`hw/chain_20261029.log`
453 帧分析	`replay/analyze453.log`
PS 决策对比	`replay/main_loop_log_20261029.jsonl`
演示视频（板子输出）	`replay/demo_board.mp4`（453 帧，15 s）
时序报告	`kv_build_int6/timing_summary.rpt`
资源报告	`bd_wrapper_utilization_placed.rpt`
迁移记录	`迁移记录-20261030-2023.2到2026.1.md`
板级细节（13 坑）	`配送小车-净空核上板-交接-20261029.md`
大模型协作	`大模型协作与技能包.md` + `codex_export/`

附录 B：大模型协作与技能包

B.1 协作方式

本项目的代码、调试、文档起草由大模型承担主要工作量；**方向、约束、验收标准由人确定，错误由人发现并纠正**。完整原始记录在代码包 `docs/codex_export/`（31 段会话、176 条提问），逐句可核。

B.2 决策权分布（186 条用户消息）

类型	条数	占比
人主动下指令 / 设约束 / 纠错 / 提供实测数据	121	65%
模型弹出选择题、人拍板	22	12%
授权执行（"继续工作"）	21	11%
其他	22	12%

B.3 模型做错方向、被人纠正的三次

日期	模型的错误做法	人的判断（原话节选）	纠正后
10-03	为了让演示达标，直接生成"看起来对"的视频	「这些都是基于算法计算出来的结果啊，**不可以你自己根据我的需求生成我想要的效果的视频**」	此后所有演示画面均取自板子写进 DDR 的原始数据
10-03	按"离障碍越远越安全"实现绕行，绕行距离过大	「一旦改变方向，意味着小车到达的时间会变，预测的落点也会改变，**此时就不用绕行那么远**」	重做时间与落点的耦合计算
10-06	连续四天按监控视角设计算法	「**我们肯定是摄像头装在小车上面啊，难道现在的算法都是以监控视角来训练的吗**」	算法改为车载视角，现场拍摄同步调整

另有一次是**人主动质疑自己的方案**：

> 「世界模型的这种预测、学习功能，**我们有点担心评委会看出来我们其实没有做世界模型的相关内容只是蹭热度**」

这条追问导致技术路线最终收窄为"动作条件预测的硬件化"，感知层如实标注为标记替代（§2.3）。

B.4 技能包

skill	解决什么
`fpga-accel-verify`	定点加速器 csim → cosim → 板级三层对拍的验收纪律；"结果偏一个常数""参数写不进""实测比估计慢"这类只有上板才暴露的问题如何定位
`amd-fpga-mcp-toolchain`	AMD 官方 agent 工具链（26 个 skill + Vivado MCP server）的部署流程与踩坑记录

`fpga-accel-verify` 与官方 26 个 skill 的分工：官方面向**设计阶段**（怎么写、怎么收敛时序），
本项目沉淀的面向**验收阶段**（怎么证明板上算的和模型一致）。它强制一个顺序：
先 csim、再 cosim、再上板，每层都要有参考值，不允许跳层。

B.5 工具链部署实测

AMD ROSS（官方 agent 工具链）已部署并实测：Vivado MCP server `2026.9.1` 注册到两个客户端，
驱动 Vivado 取回真实器件列表（`get_parts xck26*` → 2 个型号）。
部署中发现一个缺陷：路径探测会选中"存在但器件数据不全"的安装（`D:/2026.1/Vivado/bin` 缺 zynq/zynquplus，`get_parts` 返回空），已定位并给出修法。详见 `docs/AMD-ROSS-部署实录与心得.pdf`。